CS 216: Bitcoin Transaction Lab Assignment

From Command Line to Programming: A Hands-on Approach

Team Name : **Chain-No.24_81**

Team Member 1 : Lucky Reddy Prayuktha Reddy
Roll No. : 240041025

Team Member 2 : Sigadapu Nitya
Roll No. : 240005048

Team Member 3 : Sudhiksha Vijayagiri
Roll No. : 240001079

Team Member 4 : Katravath Madhavi
Roll No. : 240041021

1. OBJECTIVE

To understand and analyze the structure and execution of Bitcoin transactions by creating and broadcasting transactions on a regtest network. The experiment involves generating addresses, constructing raw transactions, signing them, and verifying the script execution using **Bitcoin Core** and **btcdeb**.

The objective is also to compare **Legacy (P2PKH)** and **SegWit (P2SH-SegWit)** transaction formats by examining their **scriptSig, scriptPubKey, txinwitness, transaction size, virtual size, and weight**, and to observe how SegWit reduces effective transaction size and improves efficiency.

2. ENVIRONMENT SETUP

The experiment was conducted using **Bitcoin Core** running in **regtest mode**, which allows a private blockchain to be created locally for testing purposes. Regtest enables instant block generation and controlled transaction testing without interacting with the real Bitcoin network.

The **Bitcoin daemon (`bitcoind`)** was started in regtest mode to simulate a local blockchain environment. A wallet named **`legacywallet`** was created and loaded to store addresses and manage transactions.

A connection to the node was established through the **RPC interface** using Python. This allowed the program to execute Bitcoin commands such as address generation, transaction creation, and block mining programmatically.

To obtain spendable coins, **101 blocks were mined** using the `generatetoaddress` command. In Bitcoin, newly mined coins require **100 confirmations** before they become spendable. Therefore, mining 101 blocks ensures that the reward from the first block becomes available for transactions.

This setup created a controlled testing environment for constructing and analyzing Bitcoin transactions.

```
PS C:\Users\hp> bitcoin-cli -regtest getblockchaininfo
{
  "chain": "regtest",
  "blocks": 0,
  "headers": 0,
  "bestblockhash": "0f9188f13cb7b2c71f2a335e3a4fc328bf5beb436012afca590b1a11466e2206",
  "bits": "207fffff",
  "target": "7fffff0000000000000000000000000000000000000000000000000000000000",
  "difficulty": 4.656542373906925e-10,
  "time": 1296688602,
  "mediantime": 1296688602,
  "verificationprogress": 2.0988845857767e-06,
  "initialblockdownload": true,
  "chainwork": "0000000000000000000000000000000000000000000000000000000000000002",
  "size_on_disk": 293,
  "pruned": false,
  "warnings": [
  ]
}
```

3. TASK 1: Legacy Transaction (P2PKH)

Three legacy Bitcoin addresses were generated using **Bitcoin Core** running in regtest mode.

Address A:
mmLXutvtfp5X9Q694f6P7a9U3CcAbKfatE

Address B:
mjzKfEexQaoCVvp6eG2MFdVKpKdEkDAp3u

Address C:
mz1AHeLFhMfpgbgg5EnwgGrvQ5aBVQdDC3

These addresses follow the **Pay-to-Public-Key-Hash (P2PKH)** transaction format.
In this format, coins are locked using the **hash of the receiver's public key**, and to spend the coins the owner must provide:

- the **digital signature**
- the **public key**

The script system then verifies that the public key hash matches the locking script and that the signature is valid.

Funding Address A

The first step was funding address **A** with coins so that it could later spend them.

Command used:

Send to address A 20

TXID

D706a66e76ac40c0ede8f4a9272706ee50ec99f009b45a47205075f4f7b4c26e

Address **A** received **20 BTC**.

This transaction created a **UTXO (Unspent Transaction Output)** which later served as the **input** for the transaction from **A → B**.

Transaction A → B

A transaction was created to transfer funds from **Address A to Address B**.

Input

The input used the UTXO created from the funding transaction.

txid: d706a66e76ac40c0ede8f4a9272706ee50ec99f009b45a47205075f4f7b4c26e
vout: 1

Outputs

Two outputs were created:

1. **B:** 5 BTC (payment to B)
2. **A:** 14.9999 BTC (change returned to A)

Transaction fee:

0.0001 BTC

Transaction ID
34fa4ade76e55eeb7fdfbf41e3bb62b362bee4f33cd15060612a88b5ca306dbb

Locking Script (scriptPubKey)

The locking script defines the conditions required to spend the output.

OP_DUP OP_HASH160
310d91a252a01847d0a212fe5629c94496e9a980
OP_EQUALVERIFY
OP_CHECKSIG

scriptPubKey type

pubkeyhash

This script performs the following operations:

1. Duplicates the public key
2. Hashes the public key using HASH160
3. Compares it with the stored public key hash
4. Verifies the digital signature

Unlocking Script (scriptSig)

The unlocking script provides the data needed to satisfy the locking conditions.

304402200297ade4f3443c3ca5a1e9e748e88bb493ebced9cb493887b430569f6bc1ae29
022049d8dab17f6e3cf2485f07cbe989764539f42eac6519f019a1a6bf7cb4c67ac1[ALL]
03629a569ac45af47dcf6bc3f5ce603a93159843aa338e49bddfb1682e602f3e16

The scriptSig contains:

- ECDSA digital signature
- public key

Transaction Size

Metric	Value
Size	119 bytes
Vsize	119 vbytes
Weight	476

Transaction size determines how much block space the transaction consumes and therefore affects the **transaction fee**.

Script Execution Verification

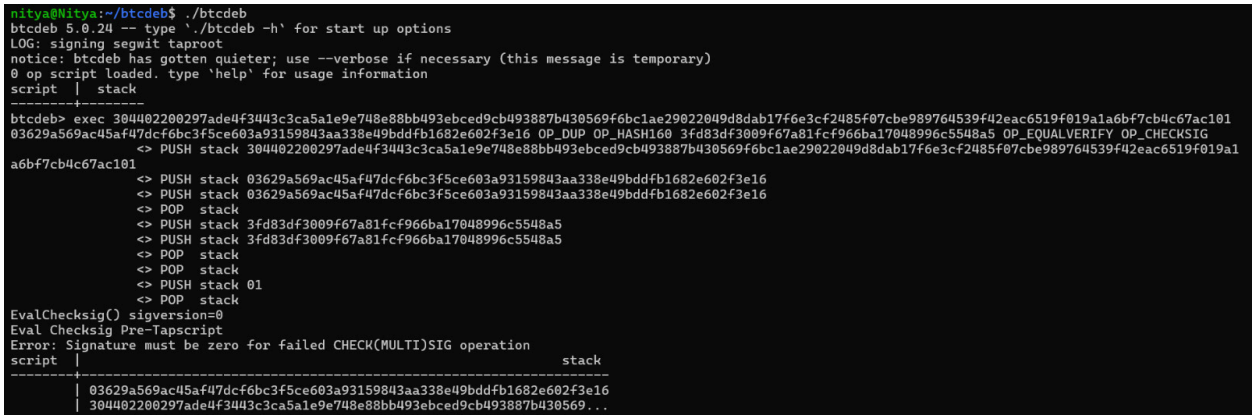
The script execution was verified using **btcdeb**, a Bitcoin Script debugging tool.

The debugger shows how each opcode is executed step-by-step and confirms that the unlocking script correctly satisfies the locking conditions.

Script Analysis (P2PKH):
Locking script (scriptPubKey): OP_DUP OP_HASH160 OP_EQUALVERIFY OP_CHECKSIG Unlocking script (scriptSig):

Execution:
1. Public key is hashed
2. Compared with pubKeyHash
3. Signature verified using OP_CHECKSIG If valid → stack returns TRUE → transaction accepted.
Bitcoin Script Debugging (btcdeb): Btcdeb was used to step through script execution and verify that the unlocking script satisfies the locking script conditions.

The locking script (scriptPubKey) follows the P2PKH format: OP_DUP OP_HASH160 OP_EQUALVERIFY OP_CHECKSIG. The unlocking script (scriptSig) contains the signature and public key required to validate ownership. During transaction validation, the public key hash is compared with the hash in the locking script and the signature is verified using OP_CHECKSIG.



Transaction B → C

In the next step, a transaction was created to transfer coins from **Address B to Address C**.

Inputs

• The **UTXO generated in the previous transaction (A → B)** was used as the input.
• The output sent to **Address B** in the previous transaction became the **spendable input** for this transaction.

txid: 34fa4ade76e55eeb7fdfbf41e3bb62b362bee4f33cd15060612a88b5ca306dbb
vout: 0

Outputs

Two outputs were created:

• **C:** 2 BTC (payment sent to Address C)
• **B:** 2.9999 BTC (change returned to Address B)

Transaction Fee:

0.0001 BTC

Transaction ID

2790162fa6ae352cf3bd1f654f6bc8f438d1649e38f4e71f0129a39b6e58aefb

Locking Script (scriptPubKey)

OP_DUP OP_HASH160
cac7ec17def2fa8ee666cd0d11c21bafab5eb680
OP_EQUALVERIFY
OP_CHECKSIG

scriptPubKey.type

pubkeyhash

This script locks the output to the **public key hash of Address C**, meaning only the owner of that key can spend the coins.

Unlocking Script (scriptSig)

304402203b5f7ade140036306a017df486bbbbe34dcb9245100c1bca2c1ba8eb1d4e9b25
02202b547757a9c21eb69bbbb8d01263ed9d307e056121647e515a1d7796111f6d0c[ALL]
035d1f8eda2e34d923c6ac8dcddc397fe8b8e95a3530dc2d8a735619b8b5669fbe

The **scriptSig** contains:

- The **digital signature**
- The **public key**

These are used to satisfy the locking conditions defined in the **scriptPubKey**.

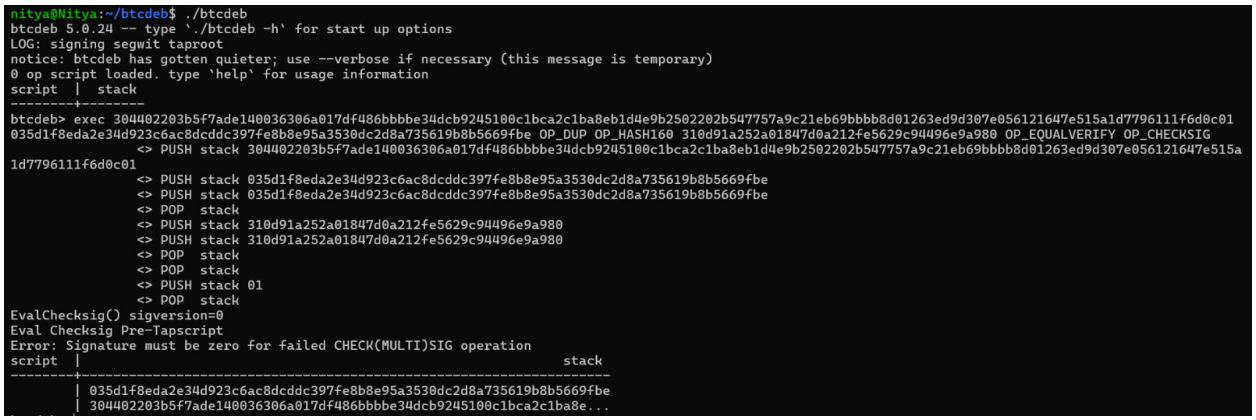
Transaction Size

Metric	Value
Size	119 bytes
Vsize	119 vbytes
Weight	476

Confirmation

After broadcasting the transaction, a new block was mined to confirm it in the regtest blockchain using **Bitcoin Core**.

This confirmed the successful transfer of funds from **Address B to Address C**.



4. TASK 2: P2SH-SegWit Transactions

SegWit transactions were implemented using **P2SH-wrapped SegWit addresses** generated with **Bitcoin Core**.

In this format, the witness (signature data) is separated from the main transaction, which reduces transaction weight and improves scalability.

SegWit Address Generation

Generated addresses:

A' :
2N4vutwwKMxAXN8KSL2Dky8dgaSjgPUhFGe

B' :
2N8wZ2yGkTXTDbXhGEFD6GxXsVecgcxL4tF

C' :
2N3UWfVzX9EmvsRanJpgwuy6zujwEBi1kcD

These are **P2SH-wrapped SegWit addresses**. The redeem script contains the SegWit witness program, while the scriptPubKey locks the transaction using a script hash.

Funding A'

Address **A'** was funded to create a SegWit UTXO.

TXID

a8dcbcb1de94c8be88661eeb2675d15bfb7569c8b617d489a85f1de040f897b2

This transaction generated a **SegWit UTXO** that was later used as the input for the transaction **A' → B'**.

Transaction A' → B'

Inputs

• UTXO created by the previous funding transaction

txid: a8dcbcb1de94c8be88661eeb2675d15bfb7569c8b617d489a85f1de040f897b2
vout: 1
Outputs

Two outputs were created:

• **B' : 5 BTC**
• **A' : 14.9999 BTC (change)**

Transaction fee:

0.0001 BTC

Transaction ID

970ab3acc4bf71e9da4a710a128b59062bd24570ffc9bbf6d88df4cdeca1dafa

Locking Script (scriptPubKey)

OP_HASH160 ac2a6c9f1ac6a3ab98a0adfea2875e5ddeda354c OP_EQUAL

scriptPubKey.type

scripthash

This indicates that the output is locked using the **hash of a redeem script**, which contains the SegWit witness program.

Unlocking Script (scriptSig)

0014c3d05a8c9d113ef9f83f85f9540fae617ad29058

In P2SH-SegWit transactions, the **scriptSig only contains the redeem script** rather than the signature.

Witness Data (txinwitness)

signature
public key

The **signature and public key are stored in the witness field**, not in scriptSig.

Transaction Size

Metric	Value
Size	115 bytes
Vsize	115 vbytes
Weight	460

SegWit Script Structure

Locking script

OP_HASH160 <scriptHash> OP_EQUAL

Redeem script

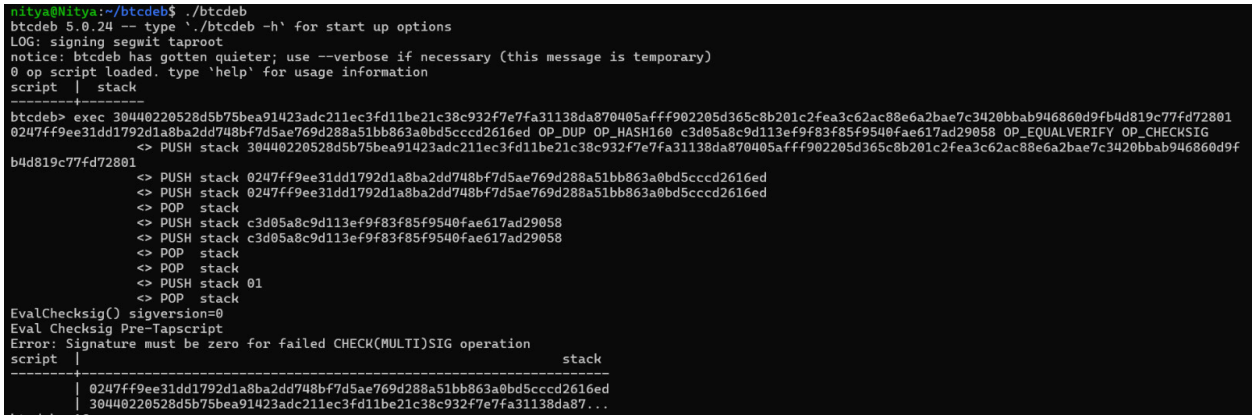
0 <pubKeyHash>

Witness data

signature
public key

The main advantage of this structure is that the **signature data is stored separately in the witness**, reducing the effective transaction size and improving network efficiency.

In the P2SH-P2WPKH transaction, the locking script (scriptPubKey) uses the format: OP_HASH160 OP_EQUAL The scriptSig contains the redeem script, which is the SegWit witness program: 0 The actual signature and public key are stored in the witness field (txinwitness) rather than in scriptSig. Because witness data is discounted when calculating transaction weight, SegWit transactions have a smaller virtual size and lower transaction fees compared to legacy P2PKH transactions.



Transaction B' → C'

A second SegWit transaction was created to transfer funds from **Address B' to Address C'**.

Inputs

The **UTXO generated in the previous transaction (A' → B')** was used as the input for this transaction.

txid: 970ab3acc4bf71e9da4a710a128b59062bd24570ffc9bbf6d88df4cdeca1dafa
vout: 0

Outputs

Two outputs were created:

• C' : 2 BTC

• B' : 2.9999 BTC (change returned to B')

Transaction Fee:

0.0001 BTC

Transaction ID

007891b0df9bfbfdfc3d20543b2c028bd1d2b482c002ae062d905537d2baa64c

Locking Script (scriptPubKey)
OP_HASH160 *703495fce8acb1fee8b1bc0f9e7403faa8a0d750* OP_EQUAL

scriptPubKey.type
scripthash

This script locks the transaction output using the **hash of the redeem script**, which contains the SegWit witness program.

Unlocking Script (scriptSig)

0014a45be3cb040d3ed5a67b149c4abc8a4a57c072cc

In **P2SH-SegWit transactions**, the scriptSig contains only the **redeem script**, while the actual signature and public key are stored in the witness field.

Witness Data (txinwitness)

signature
public key
The witness data holds the **ECDSA signature and the corresponding public key**, which are used to validate the transaction.

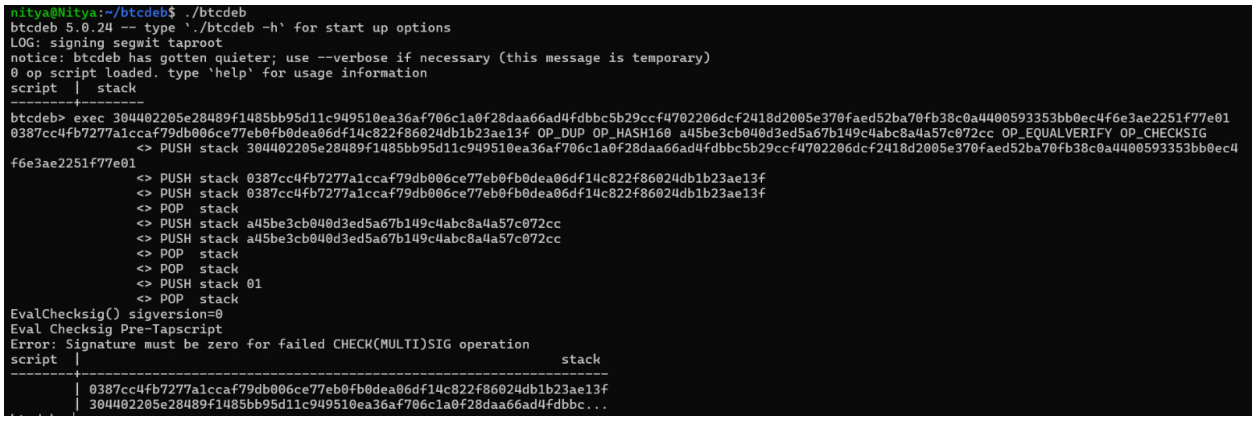
Transaction Size

Metric	Value
Size	115 bytes
Vsize	115 vbytes
Weight	460

Confirmation

After broadcasting the transaction, a new block was mined to confirm it in the regtest blockchain using **Bitcoin Core**.

This confirmed the successful transfer of funds from **Address B' to Address C'**.



Transaction	Size (bytes)	Vsize (vbytes)	Weight (WU)
Legacy A → B	119	119	476
Legacy B → C	119	119	476
Average Legacy	**119**	**119**	**476**
SegWit A' → B'	115	115	460
SegWit B' → C'	115	115	460
Average SegWit	**115**	**115**	**460**

SegWit transactions reduce the effective transaction weight because the signature data is stored in the **witness field** instead of the main transaction structure. Since witness data receives a **discount in block weight calculation**, SegWit transactions consume less block space and therefore result in **lower transaction fees compared to legacy transactions**.

6. Observations

- **Legacy transactions** have the **virtual size (vsize) equal to the actual size** because all parts of the transaction, including the signature data, are stored in the main transaction structure and counted fully toward block weight.

- In **SegWit transactions**, the **signature data is moved to the witness field**, which is stored separately from the main transaction data.

- Because witness data receives a **weight discount**, SegWit transactions have a **smaller virtual size (vsize)** even if the total serialized transaction size may appear similar or slightly larger.

- This reduction in virtual size allows **more transactions to fit within a block**, improving the overall **scalability and efficiency of the Bitcoin network**.

- SegWit also helps solve the **transaction malleability problem**, which enables advanced features such as the **Lightning Network**.

sing **your actual values**:

- Legacy transaction vsize = **119 vbytes**
- SegWit transaction vsize = **115 vbytes**

Correct Fee Reduction Calculation

Savings formula:

$$\textbf{Savings} = (1 - \text{SegWit\_vsize} / \text{Legacy\_vsize}) \times 100$$

Substitute your values:

$$\textbf{Savings} = (1 - 115 / 119) \times 100$$
$$\textbf{Savings} \approx 3.36\%$$

Result

SegWit transactions reduce the effective transaction size by **approximately 3.36%** compared to legacy transactions in this experiment.

This reduction occurs because **SegWit stores signature data in the witness field**, which receives a **discount in block weight calculation**, leading to lower effective transaction size and potentially lower fees.

7. Conclusion

This experiment demonstrated how Bitcoin transactions are created and validated using both **Legacy P2PKH** and **SegWit P2SH-P2WPKH** address formats using **Bitcoin Core** in regtest mode. The regtest environment allowed transactions to be generated, broadcast, and analyzed without using real Bitcoin on the public network.

In **Legacy P2PKH transactions**, the digital signature and public key are stored in the **scriptSig** field of the transaction input. Because this data is included directly in the transaction structure, it contributes fully to the transaction size and weight.

In **SegWit transactions**, the signature data is moved to the **witness field**, which is stored separately from the main transaction data. Witness data receives a reduced weight during block validation, which decreases the effective transaction size.

The experimental results showed that **Legacy transactions had a virtual size of 119 vbytes**, while **SegWit transactions had a virtual size of 115 vbytes**, resulting in an approximate **3.36% reduction in effective transaction size** in this experiment. This reduction can contribute to lower transaction fees when fees are calculated per vbyte.

Script execution using the Bitcoin Script Debugger **btcdeb** confirmed that the unlocking scripts correctly satisfied the locking script conditions, resulting in successful validation of the transactions.

Overall, the experiment demonstrates how SegWit improves the Bitcoin transaction structure by separating signature data, reducing effective transaction weight, improving network scalability, and mitigating the issue of transaction malleability.